Database Management System (DBMS) - Study Notes

Unit 1: Introduction to DBMS

A **database** is an organized collection of related data. A **Database Management System (DBMS)** is software that lets users create, store, update, retrieve and manage that data safely and efficiently. Examples of DBMS software are MySQL, PostgreSQL, Oracle and SQLite.

Why use a DBMS instead of plain files?

- **Reduced data redundancy:** the same data is not stored again and again in many files.
- **Data consistency:** a change made in one place is reflected everywhere.
- **Data security:** access can be controlled with user accounts and permissions.
- **Concurrent access:** many users can work on the data at the same time without conflicts.
- **Backup and recovery:** data can be restored after a crash or failure.

Three-schema architecture

The three-schema architecture separates the database into three levels. The **physical (internal) level** describes how data is actually stored on disk. The **logical (conceptual) level** describes what data is stored and the relationships between them. The **view (external) level** shows each user only the part of the data that is relevant to them. This separation gives **data independence**, which means changes at one level do not force changes at the other levels.

Types of database users

- **Database Administrator (DBA):** manages the database, security, backups and performance.
- **Application programmers:** write programs that interact with the database.
- **End users:** use the database through applications, forms and reports.

Unit 2: ER Model and Keys

The **Entity-Relationship (ER) model** is a way to design a database visually before building it. It uses three main building blocks: entities, attributes and relationships.

Basic concepts

- **Entity:** a real-world object that can be identified, such as a Student or a Course.
- **Attribute:** a property of an entity, such as roll_number, name or email.
- **Relationship:** an association between entities, for example a Student enrolls in a Course.
- **Cardinality:** describes how many instances can be linked: one-to-one, one-to-many or many-to-many.

Types of keys

Key	Meaning
Super key	Any set of attributes that uniquely identifies a row.
Candidate key	A minimal super key. No attribute can be removed from it.
Primary key	The candidate key chosen to identify rows. It cannot be NULL and must be unique.
Foreign key	An attribute in one table that refers to the primary key of another table. It links the two tables
Composite key	A key made of two or more attributes taken together.

Example: In a Student table, roll_number is the primary key. In an Enrollment table, student_roll can be a foreign key that refers to roll_number in the Student table. This guarantees that every enrollment belongs to a real student (referential integrity).

Unit 3: Normalization

Normalization is the process of organizing the tables of a database to reduce data redundancy and avoid update, insert and delete anomalies. It is done by splitting large tables into smaller related tables and linking them with keys.

Anomalies that normalization prevents

- **Insertion anomaly:** you cannot add some data without adding unrelated data.
- **Update anomaly:** the same fact must be changed in many rows, and missing one creates inconsistency.
- **Deletion anomaly:** deleting a row accidentally removes other useful information.

Normal forms

Normal form	Rule
1NF	Every column holds atomic (single) values. No repeating groups or lists inside a cell.
2NF	The table is in 1NF and has no partial dependency: every non-key attribute depends on the wh
3NF	The table is in 2NF and has no transitive dependency: a non-key attribute must not depend on
BCNF	A stronger form of 3NF. For every functional dependency X -> Y, X must be a super key.

Functional dependency: if the value of attribute A decides the value of attribute B, we write A -> B. For example roll_number -> student_name.

Trade-off: highly normalized tables reduce redundancy but need more joins when reading data. Some systems intentionally **denormalize** to make reads faster.

Unit 4: SQL Basics and Joins

SQL (Structured Query Language) is the standard language used to work with relational databases. SQL commands are grouped by purpose.

Category	Full form	Commands
DDL	Data Definition Language	CREATE, ALTER, DROP, TRUNCATE
DML	Data Manipulation Language	INSERT, UPDATE, DELETE
DQL	Data Query Language	SELECT
DCL	Data Control Language	GRANT, REVOKE
TCL	Transaction Control Language	COMMIT, ROLLBACK, SAVEPOINT

A basic query

```
SELECT name, marks FROM students WHERE marks > 60 ORDER BY marks DESC;
```

Types of joins

A **join** combines rows from two or more tables based on a related column.

- **INNER JOIN:** returns only the rows that have matching values in both tables.
- **LEFT JOIN:** returns all rows from the left table and the matching rows from the right table. Unmatched right-side values are NULL.
- **RIGHT JOIN:** returns all rows from the right table and the matching rows from the left table.
- **FULL OUTER JOIN:** returns all rows from both tables, with NULL where there is no match.
- **CROSS JOIN:** returns every possible pair of rows (the Cartesian product).

Aggregate functions: COUNT, SUM, AVG, MIN and MAX. They are often used with GROUP BY, and HAVING is used to filter groups (WHERE filters rows before grouping).

Unit 5: Transactions and ACID Properties

A **transaction** is a group of database operations that must be treated as one single unit of work. Either all the operations succeed, or none of them take effect. A classic example is transferring money: debit from account A and credit to account B must both happen together.

ACID properties

Property	Meaning
Atomicity	All operations of a transaction are completed, or none are. There is no partial result.
Consistency	A transaction takes the database from one valid state to another valid state, respecting all rules
Isolation	Transactions running at the same time do not interfere with each other. Each behaves as if it ra
Durability	Once a transaction is committed, its changes are permanent, even if the system crashes afterw

Transaction states

A transaction moves through these states: **Active** (running), **Partially committed** (final operation executed), **Committed** (changes saved permanently), **Failed** (cannot continue) and **Aborted** (rolled back to the original state).

Concurrency problems

- **Dirty read:** reading data written by another transaction that has not been committed yet.
- **Lost update:** two transactions update the same data and one update overwrites the other.
- **Non-repeatable read:** reading the same row twice in one transaction gives different values.

Databases prevent these problems using **locking** and by choosing a suitable **isolation level**.